Creating a Maven Project with Guava and Commons-IO

Google Guava Dependency

Guava is a set of core libraries developed by Google to fill gaps in Java's standard libraries. It is used for:

- Collections enhancements: Multimap, BiMap, Table, ImmutableList, etc.
- Functional programming utilities: Predicates, Functions, Suppliers.
- Caching: Built-in caching mechanisms (CacheBuilder) like a mini in-memory cache.
- String utilities: Splitter, Joiner, CharMatcher.
- Concurrency: ListenableFuture, Service, and other helper classes.
- Hashing and hashing strategies: Useful for consistent hashing (e.g., Bloom filters).

Overall, Guava simplifies and improves many core Java patterns and utilities.

Apache Commons IO Dependency

Apache Commons IO is focused purely on input/output utilities, especially for file and stream handling. It is used for:

- File operations: Easily read/write files with FileUtils.
- Stream utilities: IOUtils.copy(), IOUtils.toString() — simplifies stream handling.
- Directory monitoring: FileAlterationObserver for detecting changes in a folder.
- File filtering: WildcardFileFilter, SuffixFileFilter.

Overall, Commons IO reduces boilerplate and errors when working with files or streams.

Project 2:-Maven Project with Guava and Commons-IO Dependencies

To begin, open a terminal and generate the Maven project using the command below:

mvn archetype: generate -DgroupId=com.example -DartifactId=MyMavenGuavaApp -

DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

```
vboxuser@Ubuntu: ~/Devops
vboxuser@Ubuntu:~/Devops$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenGuavaApp -DarchetypeArtifactId=maven-archetype-quickstart -Dinterac
tiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< org.apache.maven:standalone-pom >-------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] ----------------------------------------------------------------------------
[INFO] Parameter: basedir, Value: /home/vboxuser/Devops
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenGuavaApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/vboxuser/Devops/MyMavenGuavaApp
[INFO] ----------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ----------------------------------------------------------------------------
[INFO] Total time:  1.029 s
[INFO] Finished at: 2026-03-17T19:58:42+05:30
[INFO] ----------------------------------------------------------------------------
vboxuser@Ubuntu:~/Devops$
```

Once the project is created, move into the project folder:

```
vboxuser@Ubuntu:~/Devops$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenGuavaApp -DarchetypeArtifactId=maven-archetype-quickstart -Dinterac
tiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< org.apache.maven:standalone-pom >-------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO] ----------------------------------------------------------------------------
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] ----------------------------------------------------------------------------
```

Run the tree command to confirm the directory layout that Maven generated:

```
vboxuser@Ubuntu: ~/Desktop/Devops/Lab1/MyMavenGuavaApp
vboxuser@Ubuntu:~/Desktop/Devops/Lab1/MyMavenGuavaApp$ tree
.
├── destination.txt
├── pom.xml
├── source.txt
├── src
│   ├── main
│   │   └── java
│   │       └── com
│   │           └── example
│   │               └── App.java
│   └── test
│       └── java
│           └── com
│               └── example
│                   └── AppTest.java
└── target
    ├── classes
    │   └── com
    │       └── example
    │           └── App.class
    ├── generated-sources
    │   └── annotations
    ├── generated-test-sources
    │   └── test-annotations
    ├── maven-archiver
    │   └── pom.properties
    ├── maven-status
    │   └── maven-compiler-plugin
    │       ├── compile
    │       │   └── default-compile
    │       │       ├── createdFiles.lst
    │       │       └── inputFiles.lst
    │       └── testCompile
    │           └── default-testCompile
    │               ├── createdFiles.lst
    │               └── inputFiles.lst
    ├── MyMavenGuavaApp-1.0-SNAPSHOT.jar
    ├── surefire-reports
    │   ├── com.example.AppTest.txt
    │   └── TEST-com.example.AppTest.xml
    └── test-classes
        └── com
            └── example
                └── AppTest.class

28 directories, 15 files
vboxuser@Ubuntu:~/Desktop/Devops/Lab1/MyMavenGuavaApp$
```

Open the pom.xml file in any text editor and add the required dependencies inside the <dependencies> section:

```
31
32   </dependencies>
33
34   <!-- Build: Configuring plugins and build settings -->
35 <build>
36 <plugins>
37 <!-- Example: Maven Compiler Plugin to compile Java code -->
38 <plugin>
39 <groupId>org.apache.maven.plugins</groupId>
40 <artifactId>maven-compiler-plugin</artifactId>
41 <version>3.8.1</version>
42 <configuration>
43 <source>1.7</source>
44 <target>1.7</target>
45 </configuration>
46 </plugin>
47
48 <!-- Example: Maven Surefire Plugin to run tests -->
49 <plugin>
50 <groupId>org.apache.maven.plugins</groupId>
51 <artifactId>maven-surefire-plugin</artifactId>
52 <version>2.22.2</version>
53 </plugin>
54
55
56 <plugin>
57             <groupId>org.apache.maven.plugins</groupId>
58             <artifactId>maven-jar-plugin</artifactId>
59             <version>3.1.0</version>
60             <configuration>
61                 <archive>
62                     <manifestEntries>
63                         <Main-Class>com.example.App</Main-Class>
64                     </manifestEntries>
65                 </archive>
66             </configuration>
67         </plugin>
68
69 </plugins>
70 </build>
71 </project>
```

Open pom.xml ~/DevOps/MyMavenGuavaApp

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MyMavenGuavaApp</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>MyMavenGuavaApp</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>

    <!-- https://mvnrepository.com/artifact/com.google.guava/guava -->
<dependency>
    <groupId>com.google.guava</groupId>
    <artifactId>guava</artifactId>
    <version>33.4.0-jre</version>
</dependency>

<!-- https://mvnrepository.com/artifact/commons-io/commons-io -->
<dependency>
    <groupId>commons-io</groupId>
    <artifactId>commons-io</artifactId>
    <version>2.18.0</version>
</dependency>

  </dependencies>

  <!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
```

Modify App.java

Navigate to src/main/java/com/example/ and open App.java to write the following code:

Open *App.java ~/DevOps/MyMavenGuavaApp/src/main/java/com/example Save

```java
import java.io.File;

// Import IOException to handle input-output errors
import java.io.IOException;

// Import ImmutableList from Google Guava library
// ImmutableList creates a list that cannot be modified after creation
import com.google.common.collect.ImmutableList;

// Define the main class
public class App
{
    // Main method - program execution starts from here
    public static void main( String[] args )
    {
        // Create an immutable list of strings (fruits)
        ImmutableList<String> fruits = ImmutableList.of("Apple", "Banana", "Cherry");

        // Print the list of fruits to the console
        System.out.println(fruits);

        // Create a File object representing the source file
        File sourceFile = new File("source.txt");

        // Create a File object representing the destination file
        File destFile = new File("destination.txt");

        try {
            // Copy the contents of source.txt to destination.txt
            // Using Apache Commons IO FileUtils class
            FileUtils.copyFile(sourceFile, destFile);

            // Print success message if file is copied
            System.out.println("File copied successfully!");
        }
        catch (IOException e) {
            // If any error occurs during file copying,
            // this block will handle the exception

            // Print error message
            System.err.println("Error occurred while copying file: " + e.getMessage());
        }
    }
}
```

Java Tab Width: 8 Ln 22, Col 56 INS

Compile the project by running the mvn compile command in the terminal:

```
vboxuser@Ubuntu: ~/Desktop/Devops/Lab1/MyMavenGuavaApp
vboxuser@Ubuntu:~/Desktop/Devops/Lab1/MyMavenGuavaApp$ mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------< com.example:MyMavenGuavaApp >-------------------
[INFO] Building MyMavenGuavaApp 1.0-SNAPSHOT
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenGuavaApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
```

Maven will compile all source files and place the generated JAR inside the target/ directory.

Run the Application:

If running the JAR directly throws an error, use the Maven exec plugin instead:

mvn exec:java -Dexec.mainClass=”com.example.App”

```
[Apple, Banana, Cherry]
Error occurred while copying file: Source 'source.txt' does not exist
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.509 s
[INFO] Finished at: 2026-03-17T20:13:11+05:30
[INFO] ------------------------------------------------------------------------
vboxuser@Ubuntu:~/Desktop/Devops/Lab1/MyMavenGuavaApp$
```

This may produce an error message like:

Error while copying file:Source ‘source.txt’ does not exist

To fix this, first create the required source file using:

```
Open                                                    src.txt
                                                        ~/Devops/MyMavenGuavaApp
1 My Data needs to be transferred.
```

Then re-run the application with the same exec command:

mvn exec:java -Dexec.mainClass=”com.example.App”

```
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  0.509 s
[INFO] Finished at: 2026-03-17T20:13:11+05:30
[INFO] ------------------------------------------------------------------------
vboxuser@Ubuntu:~/Desktop/Devops/Lab1/MyMavenGuavaApp$
```

This time the program runs without errors and the file is copied as expected.

Open the newly created destination.txt file to verify its contents were transferred correctly.

```
Open                                   *destination.txt
                                       ~/DevOps/MyMavenGuavaApp
1 My data needs to be transferred.
```